

18. Writing JPQL Queries using Spring Data JPA Derived Query Methods (findBy, updateBy, deleteBy, countBy)

Let's clearly understand the difference between:

1. **Predefined / Built-in Spring Data JPA Methods**
 2. **Derived Query Methods**
-

Built-in Spring Data JPA Methods

These are **already provided by Spring Data JPA** through the `JpaRepository` interface.

Example:

```
public interface UserRepository extends JpaRepository<User, Long> {  
}
```

You automatically get:

- `save()`
- `findById()`
- `findAll()`
- `deleteById()`
- `count()`
- `existsById()`

Example Usage:

```
userRepository.save(user);  
userRepository.findById(1L);  
userRepository.findAll();
```

Characteristics

- Predefined methods
 - No custom method naming needed
 - No JPQL or SQL writing
 - Works for **basic CRUD operations**
-

Derived Query Methods

These are **custom methods** where:

Spring generates the query
Based on the **method name**

You don't write JPQL or SQL,
but you **define method names carefully**.

Example Entity

```
@Entity
public class User {
    private Long id;
    private String name;
    private String email;
    private String status;
}
```

Repository with Derived Queries

```
public interface UserRepository extends JpaRepository<User, Long> {

    List<User> findByName(String name);

    void deleteByEmail(String email);

    long countByStatus(String status);

    boolean existsByEmail(String email);

    List<User> findByNameAndStatus(String name, String status);

    List<User> findByNameLike(String name);

    List<User> findByIdGreaterThan(Long id);
}
```

Spring converts them automatically into JPQL queries at runtime.

Characteristics

- Custom methods
- Query derived from method name
- No need to write @Query
- Supports:
 - And / Or
 - Between
 - Like
 - GreaterThan
 - LessThan
 - OrderBy
 - In
 - Not

- IsNull
- etc.

Common Derived Query Keywords

Keyword	Purpose
findBy	Fetch records
deleteBy	Delete records
countBy	Count records
existsBy	Check existence
And, Or	Combine conditions
Between, Like, GreaterThan	Conditions

Note: There is **no direct `updateBy` method** in derived queries. Updates require `@Modifying + @Query`.

Example Derived Query → JPQL Mapping

Method Name	JPQL Generated
findByCourse("Java")	SELECT s FROM Student s WHERE s.course = 'Java'
countByCourse("Python")	SELECT COUNT(s) FROM Student s WHERE s.course = 'Python'
deleteByEmail("a@gmail.com")	DELETE FROM Student s WHERE s.email='a@gmail.com'

Important Points

- Derived queries are converted into **JPQL**, not SQL
- Method names must follow **naming conventions**
- `updateBy` does **not exist** in derived queries
- Updates require `@Modifying + @Query`
- `@Transactional` is mandatory for delete and update queries

Project Details

File → New → Spring Starter Project →

New Spring Starter Project

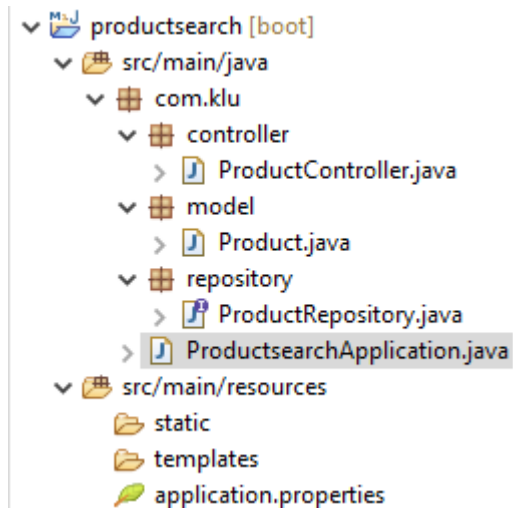


Service URL	https://start.spring.io		
Name	productsearch		
☒ Use default location			
Location	D:\SpringBoot_Projects\productsearch	Browse	
Type:	Maven	Packaging:	Jar
Java Version:	17	Language:	Java
Group	com.klu		
Artifact	productsearch		
Version	0.0.1-SNAPSHOT		
Description	Spring Mysql Deployment Project Ecommerce		
Package	com.klu		

Dependencies

- ✓ Spring Web
 - ✓ Spring Data JPA
 - ✓ MySQL Driver
 - ✓ Lombok (optional)
-

Create Spring Boot Project



MySQL Database Setup

```
=====
CREATE DATABASE productdb;
USE productdb;
=====
```

Configure **application.properties**

```
-----  
spring.datasource.url=jdbc:mysql://localhost:3306/productdb  
spring.datasource.username=root  
spring.datasource.password=root  
  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true  
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect  
  
server.port=8081  
-----
```

Create Entity Class

Product.java

```
=====
```

```
package com.klu.model;  
  
import jakarta.persistence.*;  
import lombok.*;  
  
@Entity  
@Table(name = "products")  
@Data  
@NoArgsConstructor  
@AllArgsConstructor  
public class Product {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    private String name;  
    private String category;  
    private double price;  
}
```

Following code is required **only when Lombok is not** working (**Product.java**)

```
-----
```

```
package com.klu.model;  
  
import jakarta.persistence.*;  
  
@Entity  
@Table(name = "products")  
public class Product {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long id;

private String name;
private String category;
private double price;

// No-argument constructor (required by JPA)
public Product() {
}

// All-argument constructor
public Product(Long id, String name, String category, double price) {
    this.id = id;
    this.name = name;
    this.category = category;
    this.price = price;
}

// Getters and Setters
public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getCategory() {
    return category;
}

public void setCategory(String category) {
    this.category = category;
}

public double getPrice() {
    return price;
}

public void setPrice(double price) {
```

```

        this.price = price;
    }

    // toString() method
    @Override
    public String toString() {
        return "Product{" +
            "id=" + id +
            ", name=" + name + " +
            ", category=" + category + " +
            ", price=" + price +
            '}';
    }
}

```

=====

Create Repository Interface (Derived Queries)

ProductRepository.java

=====

```

package com.klu.repository;

import com.klu.model.Product;
import org.springframework.data.jpa.repository.*;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // ----- BASIC findBy -----
    List<Product> findByCategory(String category);

    Product findByName(String name);

    // ----- And / Or -----
    List<Product> findByCategoryAndPriceGreaterThan(String category, double
price);

    List<Product> findByCategoryOrName(String category, String name);

    // ----- Between -----
    List<Product> findByPriceBetween(double min, double max);

    // ----- Like -----
    List<Product> findByNameLike(String pattern);

    // ----- GreaterThan -----
    List<Product> findByPriceGreaterThan(double price);

    // ----- countBy -----
    long countByCategory(String category);
}

```

```

// ----- existsBy -----
boolean existsByName(String name);

// ----- deleteBy -----
@Transactional
void deleteByName(String name);

// ----- JPQL -----
@Query("SELECT p FROM Product p ORDER BY p.price ASC")
List<Product> sortByPrice();
}

```

Controller Layer

ProductController.java

```

=====

package com.klu.controller;

import com.klu.model.Product;
import com.klu.repository.ProductRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/products")
public class ProductController {

    private final ProductRepository repo;

    public ProductController(ProductRepository repo) {
        this.repo = repo;
    }

    // findBy
    @GetMapping("/category/{category}")
    public List<Product> byCategory(@PathVariable String category) {
        return repo.findByCategory(category);
    }

    // And
    @GetMapping("/category/{category}/price/{price}")
    public List<Product> byCategoryAndPrice(@PathVariable String category,
                                           @PathVariable double price) {
        return repo.findByCategoryAndPriceGreaterThan(category, price);
    }

    // Or
    @GetMapping("/or")
    public List<Product> byCategoryOrName(@RequestParam String category,
                                         @RequestParam String name) {
        return repo.findByCategoryOrName(category, name);
    }

    // Between
    @GetMapping("/filter")

```

```

    public List<Product> between(@RequestParam double min,
                                @RequestParam double max) {
        return repo.findByPriceBetween(min, max);
    }

    // Like
    @GetMapping("/search")
    public List<Product> like(@RequestParam String keyword) {
        return repo.findByNameLike("%" + keyword + "%");
    }

    // GreaterThan
    @GetMapping("/expensive/{price}")
    public List<Product> greaterThan(@PathVariable double price) {
        return repo.findByPriceGreaterThan(price);
    }

    // countBy
    @GetMapping("/count/{category}")
    public long count(@PathVariable String category) {
        return repo.countByCategory(category);
    }

    // existsBy
    @GetMapping("/exists/{name}")
    public boolean exists(@PathVariable String name) {
        return repo.existsByName(name);
    }

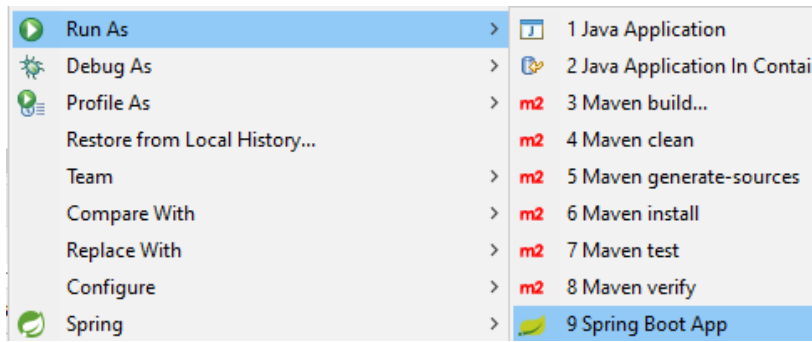
    // deleteBy
    @DeleteMapping("/delete/{name}")
    public String delete(@PathVariable String name) {
        repo.deleteByName(name);
        return "Product deleted successfully";
    }

    // JPQL sorting
    @GetMapping("/sorted")
    public List<Product> sorted() {
        return repo.sortByPrice();
    }
}
=====

```

Execution Run Spring Boot Application

- Ensure **MySQL is running**
- Database `productdb` already created

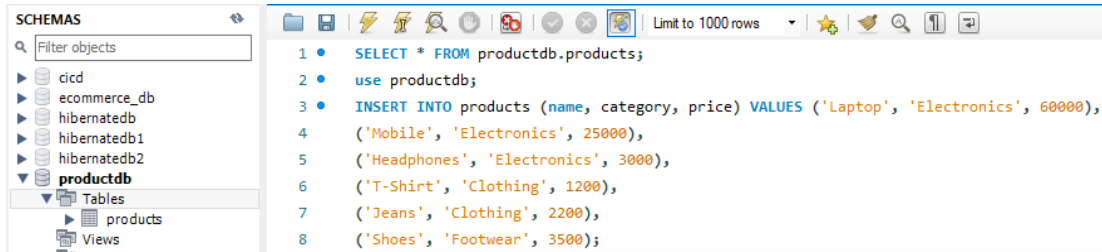


- Table “**products**” will be created **automatically** by Hibernate

Insert Sample Product Records Using MySQL

=====

```
use productdb;
INSERT INTO products (name, category, price) VALUES
('Laptop', 'Electronics', 60000),
('Mobile', 'Electronics', 25000),
('Headphones', 'Electronics', 3000),
('T-Shirt', 'Clothing', 1200),
('Jeans', 'Clothing', 2200),
('Shoes', 'Footwear', 3500);
```



=====

Execution of Derived Query Methods

Output URLs (Testing through Postman)

<http://localhost:8081/>

Operation	URL
findByCategory	/products/category/Electronics
findBy And	products/category/Electronics/price/3000
findBy Or	/products/or?category=Clothing&name=Shoes
Between	/products/filter?min=1000&max=5000

Operation	URL
Like	/products/search?keyword=Lap
GreaterThan	/products/expensive/20000
countBy	/products/count/Electronics
existsBy	/products/exists/Laptop
deleteBy	DELETE /products/delete/Laptop
JPQL sort	/products/sorted

GET http://localhost:8081/products/category/Electronics

Docs Params Authorization Headers (8) Body Scripts Set

Query Params

Key	Value
-----	-------

Body Cookies Headers (5) Test Results ↻

{} JSON Preview Visualize

```
1 {
2   {
3     "id": 1,
4     "name": "Laptop",
5     "category": "Electronics",
6     "price": 60000.0
7   },
8 }
```

GET http://localhost:8081/products/category/Electronics/price/30000

Docs Params Authorization Headers (8) Body Scripts Settings

Query Params

Key	Value
-----	-------

Body Cookies Headers (5) Test Results ↻

{} JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "name": "Laptop",
5     "category": "Electronics",
6     "price": 60000.0
7   },
8 ]
```

GET http://localhost:8081/products/or?category=Clothing&name=Shoes

Docs Params Authorization Headers (8) Body Scripts Settings

☒ name Shoes

Key	Value
-----	-------

Body Cookies Headers (5) Test Results ↻

{} JSON Preview Visualize

```
5   "category": "Clothing",
6   "price": 1200.0
7 },
8 {
9   "id": 5,
10  "name": "Jeans",
11  "category": "Clothing",
12  "price": 2200.0
13 },
14 {
15   "id": 6,
16   "name": "Shoes",
17   "category": "Footwear",
18   "price": 3500.0
19 }
```

GET http://localhost:8081/products/exists/Laptop

Docs Params Authorization Headers (8) Body Sc

Key	Value
-----	-------

Body Cookies Headers (5) Test Results ↻

{} JSON Preview Visualize

```
1 true
```

GET

http://localhost:8081/products/count/Electronics

Docs

Params

Authorization

Headers (8)

Body

Scripts

Key
Key

Body

Cookies

Headers (5)

Test Results

JSON

Preview

Visualize

```
1 3
```

DELETE

http://localhost:8081/products/delete/Laptop

Docs

Params

Authorization

Headers (8)

Body

Query Params

Key
Key

Body

Cookies

Headers (5)

Test Results

Raw

Preview

Visualize

```
1 Product deleted successfully
```

GET

http://localhost:8081/products/search?keyword=Lap

Docs

Params

Authorization

Headers (8)

Body

Scripts

keyword	Lap
Key	Value

Body

Cookies

Headers (5)

Test Results

JSON

Preview

Visualize

```
1 [
2   {
3     "id": 1,
4     "name": "Laptop",
5     "category": "Electronics",
6     "price": 60000.0
```